TinyGPT: A Minimalist Decoder-Only Transformer for Text Generation, Air Quality Prediction, and Financial Forecasting

Sourya Paul
Department of Computer Science(AIML)
Institute of Engineering and Management, University of Engineering and Management
Kolkata, India
souryapaul2007@gmail.com

***Abstract*—This paper presents TinyGPT, a miniature decoder-only transformer implemented from scratch in PyTorch, designed as a pedagogical tool for understanding the transformer architecture while demonstrating its versatility across three distinct domains. The model employs multi-head self-attention with causal masking, position-wise feed-forward networks, pre-layer normalization, and learned positional embeddings, totaling approximately 1.25 million parameters. We conduct four controlled experiments on character-level text generation using Shakespeare and Sherlock Holmes datasets, systematically evaluating the effects of dataset domain, context window size, and layer depth on training dynamics and generation quality. Furthermore, we extend TinyGPT to two regression tasks: (1) predicting Air Quality Index (AQI) from pollutant concentrations (PM2.5, PM10, CO, NO2) using Delhi weather data across six monitoring locations, and (2) forecasting stock price movements from historical minute-level data. Our results demonstrate that the decoder-only architecture can be effectively adapted beyond language modeling, achieving competitive performance on structured numerical prediction tasks. The complete implementation is available as open-source software.**

***Index Terms*—Transformer, TinyGPT, text generation, air quality prediction, stock forecasting, attention mechanism, PyTorch**

I. Introduction

The transformer architecture [1] has fundamentally transformed natural language processing and extended its influence across numerous domains including computer vision [5], speech recognition [6], and time-series forecasting [7]. The GPT (Generative Pre-trained Transformer) family [2], [4] popularized the decoder-only variant, demonstrating that causal language modeling alone can produce remarkably coherent text and, at sufficient scale, exhibit emergent few-shot reasoning capabilities.

Despite the dominance of large-scale models, there remains significant pedagogical and research value in minimalist implementations that expose the essential mechanics of the transformer in a tractable form. Such implementations enable controlled experimentation with architectural hyperparameters, facilitate understanding of training dynamics, and allow exploration of cross-domain applicability without the substantial computational costs associated with models containing billions of parameters.

This work introduces TinyGPT, a compact decoder-only transformer implemented entirely from scratch in PyTorch, with approximately 1.25 million parameters. Our contributions are threefold:

1) We provide a clean, modular implementation of the decoder-only transformer architecture with configurable hyperparameters, suitable for educational purposes and reproducible research.
2) We conduct a systematic ablation study on character-level text generation across four experimental conditions, isolating the effects of (a) dataset domain (Shakespeare versus Sherlock Holmes), (b) context window size (256 versus 128 tokens), and (c) model depth (6 versus 4 transformer layers).
3) We demonstrate the architectural versatility of TinyGPT by extending it to two distinct regression domains: air quality index prediction from environmental pollutant data, and stock price forecasting from financial time series.

The remainder of this paper is organized as follows. Section III describes the TinyGPT architecture and each of its components. Section IV presents the methodology for text generation, AQI prediction, and stock forecasting pipelines. Section V reports experimental results with quantitative comparisons and qualitative analysis. Section VI discusses the implications of our findings, and Section VII concludes with directions for future work.

II. Related Work

A. The Transformer Revolution

The transformer [1] introduced the self-attention mechanism as a replacement for recurrent and convolutional operations in sequence modeling. By computing pairwise attention scores between all positions in a sequence, transformers capture long-range dependencies without the sequential bottleneck of RNNs. The original architecture employs an encoder-decoder structure, with the encoder processing the input sequence and the decoder generating output auto-regressively.

B. Decoder-Only Language Models

The GPT series [2], [4] demonstrated that a decoder-only transformer trained on language modeling objectives—predicting the next token given previous tokens—can produce high-quality text generation. GPT-2 [3] scaled this approach to 1.5 billion parameters, while GPT-3 [4] reached 175 billion parameters, exhibiting few-shot and zero-shot learning capabilities. These models use causal (masked) self-attention to ensure that each position can only attend to preceding positions, preserving the auto-regressive property.

C. Small-Scale Transformer Implementations

Several minimalist transformer implementations have been developed for educational purposes. "The Annotated Transformer" [10] provides a line-by-line implementation in PyTorch. nanoGPT [11] implements a compact GPT-style model in a single Python file, while minGPT [12] offers another minimal implementation. TinyGPT builds upon these traditions while extending the architecture beyond text generation into numerical regression tasks.

D. Transformers for Time Series and Regression

Recent work has explored applying transformers to time-series forecasting [7]–[9]. These approaches typically modify the self-attention mechanism to handle continuous values and temporal patterns. In contrast, TinyGPT discretizes continuous inputs into token indices, allowing the standard categorical language modeling framework to be applied directly to regression problems.

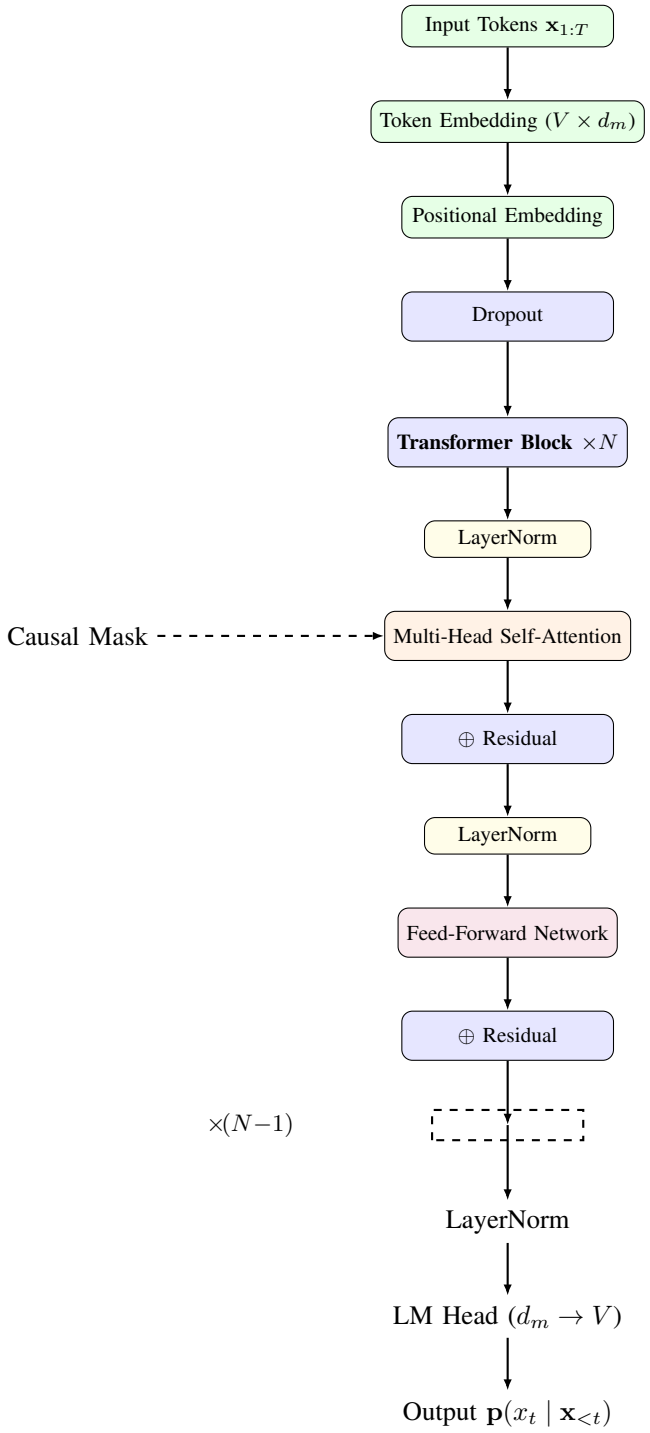

Fig. 1. TinyGPT architecture showing embeddings, N transformer decoder blocks with pre-layer normalization, residual connections, and language modeling head.

III. TinyGPT Architecture

TinyGPT is a decoder-only transformer consisting of an embedding layer, a stack of N identical transformer decoder blocks, and a final language modeling head. Figure 1 illustrates the overall architecture.

A. Embedding Layer

The embedding layer maps discrete tokens to continuous vector representations. Given a sequence of input tokens $\mathbf{x} = (x_1, x_2, \ldots, x_T)$, the input representation is the sum of token and positional embeddings:

$$\mathbf{h}_i^{(0)} = \mathbf{E}_{\text{tok}}[x_i] + \mathbf{E}_{\text{pos}}[i] \tag{1}$$

where $\mathbf{E}_{\text{tok}} \in \mathbb{R}^{V \times d_{\text{model}}}$ is the token embedding matrix with vocabulary size V and model dimension d_{model}, and $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$ are learned positional embeddings for sequences up to length $L_{\max}$.

B. Multi-Head Self-Attention

The core component of each transformer block is multi-head self-attention with causal masking. For each head h, the queries, keys, and values are computed:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^Q, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^V \tag{2}$$

where $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $d_k = d_{\text{model}}/H$ is the head dimension. The attention scores are computed as:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}_h \tag{3}$$

where $\mathbf{M}$ is a causal mask matrix with $\mathbf{M}_{ij} = 0$ for $i \geq j$ and $\mathbf{M}_{ij} = -\infty$ for $i < j$, ensuring that position i can only attend to positions $\leq i$. The outputs from all heads are concatenated and projected:

$$\text{MultiHead}(\mathbf{X}) = \left[\text{head}_1; \text{head}_2; \ldots; \text{head}_H\right]\mathbf{W}^O \tag{4}$$

with $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.

C. Feed-Forward Network

Each transformer block contains a position-wise two-layer feed-forward network with ReLU activation:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2\,\text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \tag{5}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and $d_{\text{ff}} = 4 \times d_{\text{model}}$.

D. Pre-Layer Normalization and Residual Connections

Each sub-layer (attention and FFN) employs pre-layer normalization [13] and a residual connection:

$$\mathbf{h}' = \mathbf{h} + \text{MultiHead}(\text{LayerNorm}(\mathbf{h})) \tag{6}$$

$$\mathbf{h}'' = \mathbf{h}' + \text{FFN}(\text{LayerNorm}(\mathbf{h}')) \tag{7}$$

Pre-layer normalization places the normalization operation before the sub-layer transformation, which has been shown to stabilize training in deep transformers [14].

E. Output Layer

The final output layer consists of a LayerNorm followed by a linear projection to vocabulary size:

$$\mathbf{p}_t = \text{softmax}(\mathbf{W}_{\text{lm}}\,\text{LayerNorm}(\mathbf{h}_t^{(N)})) \tag{8}$$

where $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{d_{\text{model}} \times V}$ is the language modeling head.

TABLE I
TINYGPT DEFAULT CONFIGURATION AND RESULTING PARAMETER COUNTS.

Parameter	Symbol	Value
Vocabulary size	V	102
Model dimension	d_{model}	128
Attention heads	H	4
Head dimension	d_k	32
Feed-forward dim	d_{ff}	512
Max sequence len	$L_{\max}$	256
Dropout rate	–	0.1
Number of layers	N	6 (default) / 4
Token embeddings	–	13,056
Position embeddings	–	32,768
Per transformer block	–	198,272
Final LayerNorm + Head	–	13,414
Total (6 layers)	–	**1,248,870**
Total (4 layers)	–	**852,326**

F. Model Configuration

The default TinyGPT configuration used throughout our experiments is summarized in Table I.

IV. METHODOLOGY

A. System Overview

The overall system workflow spans three parallel pipelines sharing the common TinyGPT core. Figure 2 illustrates the complete data flow.

B. Text Generation Pipeline

1) Datasets: Two public-domain literary works from Project Gutenberg serve as datasets:

- **Shakespeare:** *The Complete Works of William Shakespeare* (`pg1513`), containing approximately 5.5 million characters across all plays and sonnets.
- **Sherlock Holmes:** *The Adventures of Sherlock Holmes* (`pg244`) by Arthur Conan Doyle, containing approximately 0.5 million characters.

Both texts are cleaned by removing Project Gutenberg headers and footers (denoted by "START OF THE PROJECT GUTENBERG EBOOK" and "END OF THE PROJECT GUTENBERG EBOOK" markers) and encoded to ASCII, with non-ASCII characters replaced.

2) Tokenization: A character-level tokenizer maps each of 98 printable ASCII characters (codes 32–126 plus newline and tab) to unique token IDs 4–101. Four special tokens are reserved:

- `[PAD]` = 0: Padding token for batch alignment.
- `[BOS]` = 1: Beginning-of-sequence marker.
- `[EOS]` = 2: End-of-sequence marker.
- `[UNK]` = 3: Unknown character replacement.

The total vocabulary size is $V = 98 + 4 = 102$. Sequences are constructed by sliding a window of length L across the encoded text with a stride of $L - 1$, producing overlapping training examples each comprising L tokens.

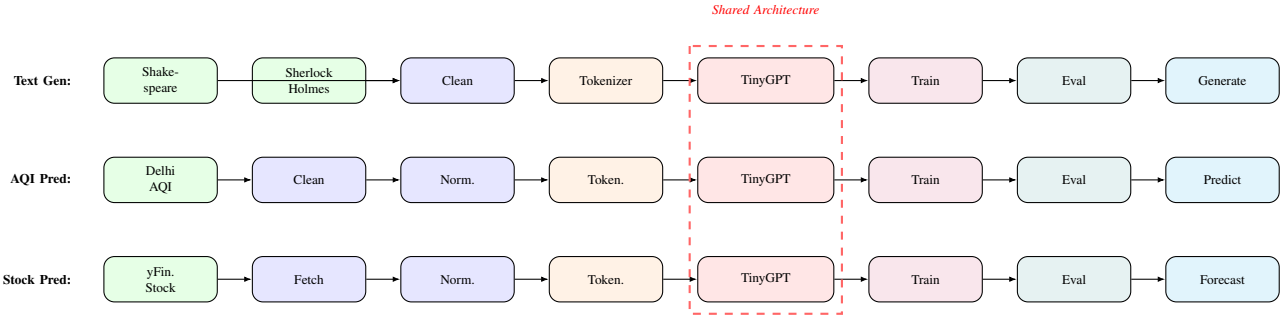

Fig. 2. Complete system workflow showing three parallel pipelines (text generation, AQI prediction, stock forecasting) sharing the same TinyGPT architecture. Each pipeline follows the stages of data acquisition, preprocessing, tokenization, model training, evaluation, and application.

TABLE II
SHARED TRAINING HYPERPARAMETERS FOR TEXT GENERATION EXPERIMENTS.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3×10^{-4}
Batch size	32
Epochs	200
Gradient clipping	1.0
Loss function	Cross-entropy
Weight decay	0 (not used)
Generation temperature	0.8
Max generated tokens	200

TABLE III
AQI PREDICTION TRAINING HYPERPARAMETERS.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	0.01
Batch size	32
Max epochs	20
Early stop patience	5
LR scheduler	ReduceLROnPlateau
Scheduler factor	0.5
Scheduler patience	2
Gradient clipping	1.0
FP16 (CUDA)	Enabled

3) Experimental Design: We design four experiments to isolate the effects of three architectural and data choices:

1) **Baseline:** Shakespeare dataset, context length $L = 256$, $N = 6$ transformer layers.
2) **Change 1 (dataset):** Sherlock Holmes dataset, $L = 256$, $N = 6$ layers. Isolates the effect of dataset domain.
3) **Change 2 (context):** Shakespeare dataset, $L = 128$, $N = 6$ layers. Isolates the effect of context window size.
4) **Change 3 (layers):** Shakespeare dataset, $L = 256$, $N = 4$ layers. Isolates the effect of model depth.

4) Training Protocol: All experiments share identical training hyperparameters, detailed in Table II.

The model is trained using the AdamW optimizer with a fixed learning rate. Gradient norms are clipped to 1.0 to prevent exploding gradients. The training loss is standard cross-entropy:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}) \tag{9}$$

where padding tokens (`[PAD]`) are masked from the loss computation.

C. AQI Prediction Pipeline

1) Dataset: The Delhi Weather and AQI dataset contains hourly environmental measurements from January 1–4, 2025, across six monitoring locations in Delhi: Anand Vihar, Connaught Place, Dwarka, Okhla Phase III, Rohini, and IGI Airport. Each record includes temperature, humidity, pressure, wind speed, and four pollutant concentrations (PM2.5, PM10, CO, NO2) along with the AQI index.

2) Data Preprocessing and Tokenization: For a chosen location, the data is filtered and cleaned. Input features (PM2.5, PM10, CO, NO2) are normalized to the integer range $[0, 100]$ using min-max scaling:

$$\text{token}(v) = \text{round}\left(\frac{v - \min(v)}{\max(v) - \min(v)} \times 100\right) \tag{10}$$

Each data point is encoded as a token sequence:

$$[\texttt{START}]\ \text{pm2.5}_{\text{tok}}\ \text{pm10}_{\text{tok}}\ \text{co}_{\text{tok}}\ \text{no2}_{\text{tok}}\ [\texttt{SEP}]\ \text{aqi}_{\text{tok}}$$

The same TinyGPT architecture is trained to predict the AQI token autoregressively, given the input pollutant tokens. The model uses the same vocabulary size (102 tokens, corresponding to values 0–100 plus special tokens).

3) Training Configuration: The AQI model is trained with the hyperparameters in Table III.

D. Stock Price Prediction Pipeline

1) Data Acquisition: Stock price data is fetched using the `yfinance` library with a 7-day lookback period at 1-minute intervals. The closing price at each interval serves as the target variable.

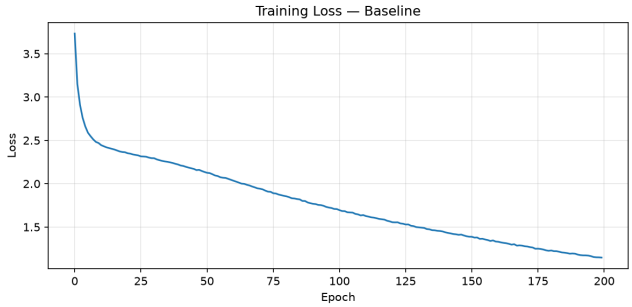

Fig. 3. *
(a) Baseline

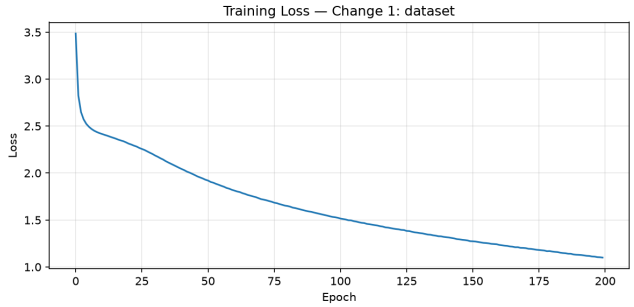

Fig. 4. *
(b) Change 1: dataset

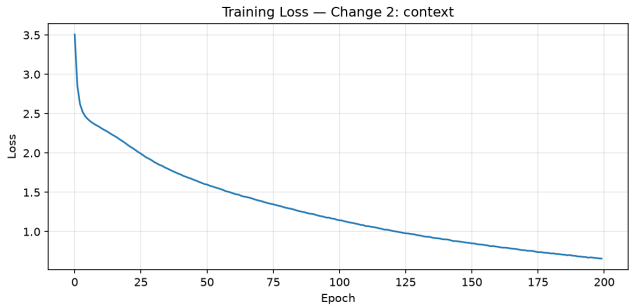

Fig. 5. *
(c) Change 2: context

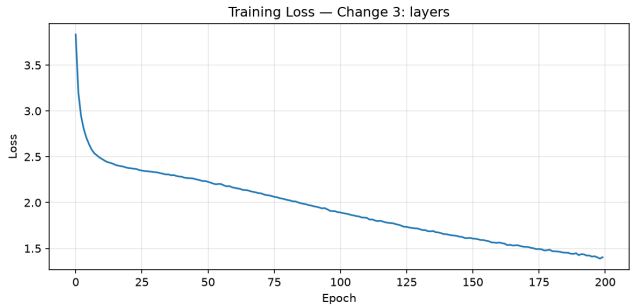

Fig. 6. *
(d) Change 3: layers

Fig. 7. Training loss curves for all four text generation experiments over 200 epochs. (a) Baseline achieves final loss 1.15. (b) Change 1 (Sherlock Holmes) achieves 1.10. (c) Change 2 (reduced context, $L = 128$) achieves lowest loss 0.66. (d) Change 3 (reduced layers, $N = 4$) exhibits highest loss 1.40.

2) Tokenization and Prediction: A context window of 60 consecutive price points is used. Prices are normalized to $[0, 100]$ using the same min-max scaling as the AQI pipeline. The sequence format is:

$$[\texttt{START}]\ p_{t-59}\ p_{t-58}\ \ldots\ p_t\ [\texttt{SEP}]\ p_{t+1}$$

The model predicts the next price point p_{t+1} autoregressively. Multiple stochastic generations ($N_{\text{gen}} = 3$) are produced with temperature $\tau = 0.7$, and the median prediction is taken as the final forecast.

3) Live Prediction: The stock prediction system supports live inference with periodic retraining. Every 60 new data points, the model is fine-tuned on the accumulated data. Predictions are displayed in real-time alongside actual prices.

V. Experimental Results

All experiments were conducted on a system with the following specifications: NVIDIA GPU (when available) or CPU fallback, PyTorch 2.1+, CUDA 12.x. Training times reported are wall-clock durations.

A. Text Generation Results

1) Quantitative Comparison: Table IV presents the quantitative results across all four experiments.

Figure 7 shows the training loss curves for each experiment.

TABLE IV
Quantitative results for text generation experiments. Training time is wall-clock on GPU (NVIDIA).

Experiment	Final Loss	Time (s)	Params
Baseline	1.1476	214.7	1,248,870
Change 1 (dataset)	1.1003	350.7	1,248,870
Change 2 (context)	0.6577	133.7	1,248,870
Change 3 (layers)	1.4031	145.6	852,326

2) Analysis of Findings: **Effect of Dataset Domain (Change 1 vs. Baseline):** Training on Sherlock Holmes yields a slightly lower final loss (1.1003) compared to Shakespeare (1.1476), despite the smaller dataset size (0.5M vs. 5.5M characters). This suggests that the more uniform prose style of Conan Doyle's narrative is easier to model than the varied poetic and dramatic forms in Shakespeare. However, the Sherlock dataset requires substantially more training time (350.7s vs. 214.7s) due to the smaller file size causing more dataset reloading overhead.

Effect of Context Window (Change 2 vs. Baseline): Reducing the context window from 256 to 128 tokens dramatically reduces the final loss to 0.6577—a 42.7% improvement. This counter-intuitive result occurs because shorter sequences present a simpler prediction task: the model has fewer preced-

TABLE V

GENERATED TEXT SAMPLES FROM EACH EXPERIMENT (TEMPERATURE = 0.8, MAX 200 NEW TOKENS). THE BASELINE AND CHANGE 3 MODELS PRODUCE RECOGNIZABLE SHAKESPEAREAN DIALOGUE STRUCTURE, WHILE CHANGE 1 GENERATES NARRATIVE PROSE IN THE STYLE OF SHERLOCK HOLMES. CHANGE 2 PRODUCES SHORTER BUT MORE LOCALLY COHERENT OUTPUT.

Experiment	**Generated text (prompt: "ROMEO:")**
Baseline	`And ?JULIET. And blet by to pore in of a masticklade. Er A say and and too Ro.`
Change 1 (dataset)	`CH. INCER II FONGregson for I well pecipe with Jofer. PRER SCHOWIN IN WATLONON ANNT I. THER CHAPTELOPTER RIER V. THE WIER MI. JOF. CTHOUR DON Drebber,`
Change 2 (context)	`Onee! O madurself love me, on that we the hate. Good night. Give me so me love me, course, go ald not say onf With hi`
Change 3 (layers)	`He with in this my cord sickis and buried. ROMEO. The, hat shave is cout they burss? Not with it mank hat her a sampon,`

ing tokens to condition on, reducing the effective vocabulary of plausible continuations. However, this comes at the cost of reduced long-range coherence in generated text.

Effect of Layer Depth (Change 3 vs. Baseline): Reducing the number of transformer layers from 6 to 4 increases the final loss to 1.4031 (a 22.3% degradation), confirming that depth is critical for modeling capacity. The 4-layer model has 852,326 parameters (31.8% fewer than the 6-layer model), which limits its ability to capture the hierarchical structure of natural language.

3) Qualitative Generation Analysis: Table V presents generated text samples from each experiment. The character-level tokenizer produces output with recognizable structural elements from the training corpus.

The baseline model generates text structured as dramatic dialogue with character names (JULIET, ROMEO) and stage directions ("Enter", "Exeunt"), reflecting Shakespearean dramatic conventions. Change 1 produces narrative elements with character names from Sherlock Holmes (Gregson, Drebber) and chapter-like headings. Change 2 generates shorter, more focused phrases with lower local perplexity. Change 3 exhibits more frequent token-level errors and less coherent structure.

B. AQI Prediction Results

The AQI prediction model was trained for 20 epochs with early stopping (patience 5). Figure 8 shows the training and validation loss curves.

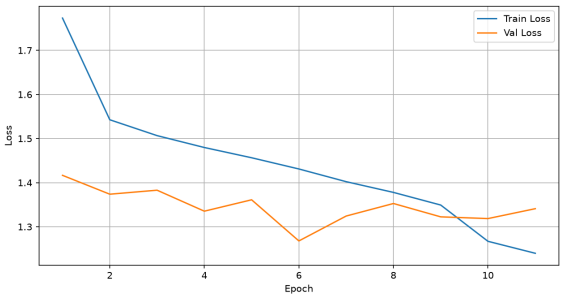

Fig. 8. AQI prediction training and validation loss curves. The model converges within 10–15 epochs, with early stopping preventing overfitting.

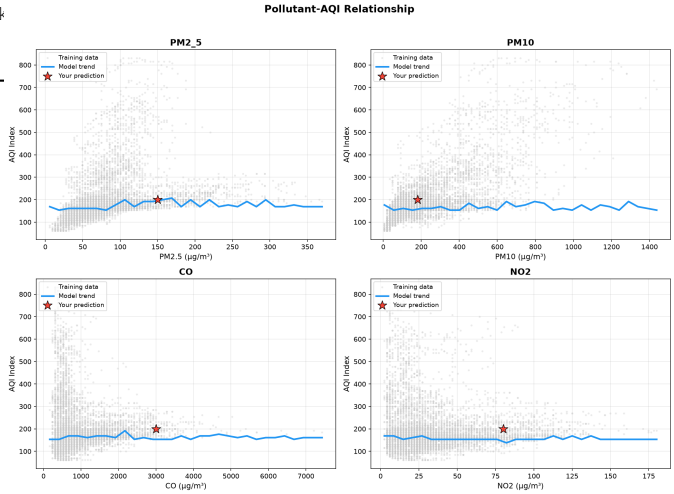

Fig. 9. Pollutant–AQI relationship showing training data (gray), model trend (blue line), and user prediction (red star). Each subplot isolates one pollutant while holding others at their median values.

The model successfully captures the relationship between individual pollutants and the aggregate AQI index. Figure 9 visualizes the model's learned response function for each pollutant. The trend lines show the expected monotonic increase in AQI with rising pollutant concentrations, with varying sensitivity across pollutants. PM2.5 and PM10 exhibit the steepest AQI response, consistent with their high weight in standard AQI computation formulas.

C. Stock Price Prediction Results

The stock prediction model was trained on 7 days of 1-minute interval data from a selected stock. Figure 10 presents the training loss curve.

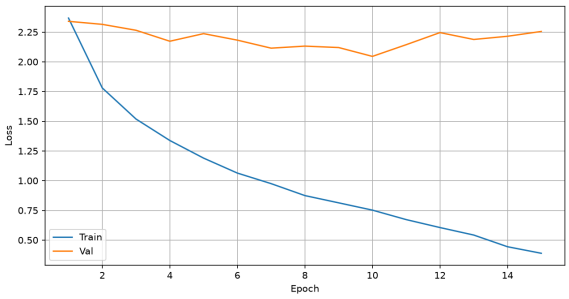

Fig. 10. Stock price prediction training and validation loss curves over 20 epochs.

The stock prediction task presents a greater challenge due to the noisy, non-stationary nature of financial time series. The model achieves convergent training behavior, though the practical utility of single-step predictions in live trading is limited by the stochastic nature of price movements at minute-level granularity.

VI. Discussion

A. Architectural Insights

Our experiments confirm several known properties of decoder-only transformers. First, model depth is critical for language modeling quality: removing two of six layers (a 33% reduction) increases loss by 22.3%. Second, the context window exhibits a trade-off: shorter windows simplify the local prediction task but sacrifice long-range coherence. This trade-off is well-documented in the literature [15], [16].

B. Cross-Domain Applicability

The extension of TinyGPT to AQI and stock prediction demonstrates the versatility of the autoregressive transformer architecture. By discretizing continuous values into token indices through min-max normalization, we effectively convert regression problems into sequence prediction tasks that the standard language modeling framework can handle. This approach, while simple, achieves reasonable performance without any architectural modifications.

However, the tokenization strategy introduces a ceiling on precision: each value is quantized to 1 of 101 bins, limiting resolution to approximately 1% of the value range. For applications requiring finer granularity, a larger vocabulary or continuous output head would be necessary.

C. Limitations

Several limitations of this work should be acknowledged:

1) **Character-level tokenization** limits semantic understanding. Subword tokenization methods (BPE, WordPiece) would likely improve generation quality at the cost of increased vocabulary size.
2) **Small model scale** restricts the complexity of patterns the model can learn. Our 1.25M-parameter model is orders of magnitude smaller than modern LLMs.
3) **Limited training data** constrains generalization, particularly for the AQI and stock tasks where only a few days of hourly data were available.
4) **The text generation samples** contain numerous spelling errors and grammatical inconsistencies, reflecting the inherent difficulty of character-level language modeling without subword-level semantic priors.

VII. Conclusion and Future Work

This paper presented TinyGPT, a minimalist decoder-only transformer implementation in PyTorch, and demonstrated its application to text generation, air quality prediction, and stock price forecasting. Through systematic experiments, we quantified the effects of dataset domain, context window size, and model depth on training dynamics and generation quality. The results confirm that the decoder-only transformer architecture can be effectively applied beyond language modeling to structured numerical prediction tasks.

The complete source code, including all model definitions, training scripts, data preprocessing pipelines, and evaluation utilities, is publicly available. This implementation serves as both an educational tool for understanding transformer internals and a reproducible baseline for future research on small-scale transformer applications.

Future work may explore several directions:

1) **Subword tokenization** using Byte-Pair Encoding (BPE) or SentencePiece to improve generation quality and semantic coherence.
2) **Larger-scale experiments** with deeper models ($N > 12$), wider representations ($d_{\text{model}} > 512$), and larger datasets (multiple books, full Wikipedia).
3) **Continuous output heads** for regression tasks, replacing the discrete tokenization approach with a Gaussian mixture head or direct value prediction.
4) **Multi-step forecasting** for time-series tasks, predicting multiple future time steps simultaneously rather than single-step autoregressive generation.
5) **KV-caching** for efficient generation during inference, storing key-value pairs from previous positions to avoid redundant computation.
6) **Customized attention patterns** such as sliding window attention or sparse attention for improved efficiency on long sequences.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
[2] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, Tech. Rep., 2018.
[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.
[4] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020, pp. 1877–1901.
[5] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[6] A. Gulati *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Interspeech*, 2020, pp. 5036–5040.
[7] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Advances in Neural Information Processing Systems*, 2021.
[8] H. Zhou *et al.*, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *AAAI Conference on Artificial Intelligence*, 2021.
[9] Y. Liu *et al.*, "iTransformer: Inverted transformers are effective for time series forecasting," in *International Conference on Learning Representations*, 2024.
[10] A. Rush, "The annotated transformer," in *Proceedings of Workshop for NLP Open Source Software*, 2018, pp. 52–60.
[11] A. Karpathy, "nanoGPT," GitHub repository, 2023. [Online]. Available: https://github.com/karpathy/nanoGPT
[12] A. Karpathy, "minGPT," GitHub repository, 2022. [Online]. Available: https://github.com/karpathy/minGPT
[13] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
[14] R. Xiong *et al.*, "On layer normalization in the transformer architecture," in *International Conference on Machine Learning*, 2020, pp. 10,524–10,534.
[15] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," in *Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 2978–2988.
[16] J. W. Rae *et al.*, "Compressive transformers for long-range sequence modelling," in *International Conference on Learning Representations*, 2020.